SpendAudit: Evaluating LLM Payment Auditors Against an Executable Policy

Ian Alloway

Working Paper · October 2026

Abstract. An LLM auditor can produce a plausible explanation while approving an action that violates policy. SpendAudit studies this failure in a setting where the correct verdict can be computed without an LLM: synthetic payment histories governed by seven explicit spending rules. The benchmark contains 160 scenarios, each presented in a clean form and in a form whose payment memo carries a prompt injection. All 320 labels were re-derived directly from the rendered traces, and each pair differs only in that memo. I compare a verdict-only protocol with one that requires rule-by-rule evidence and arithmetic, across three models. Of 1,920 intended judgments, 1,388 returned a usable verdict, 531 failed with API errors, and one was malformed. On scenarios with usable verdicts in all four conditions, the evidence protocol never made more errors than verdict-only and made fewer in five of six model–trace comparisons; on clean traces it reduced false approvals from 34 to 6 of 65 violating payments for one model and from 47 to 18 of 83 for another (exact McNemar $p < 0.001$ for both), without a single false rejection in any condition. Injected memos, however, still led the evidence protocol to approve violating payments for every model, including 10 of 66 for a model that made no errors on the corresponding clean traces. Because API failures left coverage incomplete and uneven, with boundary cases most affected, these results are descriptive rather than benchmark-level estimates. The main contribution is an evaluation design in which each auditor error can be traced to a specific rule, time window, and calculation.

Keywords: LLM evaluation; LLM-as-a-judge; prompt injection; AI auditing; agent oversight; spend controls

1 Introduction

An agent that can request payments raises two separate evaluation questions: whether it completed its task, and whether each action it took was permitted. A useful research brief does not show that the agent respected a vendor restriction, stayed within a rolling budget, or preserved a minimum cash reserve. Answering the second question requires reconstructing the relevant state from the transaction history and applying the policy at the moment of the proposed action.

LLMs are attractive auditors for this work because real spending policies often contain clauses that resist exact encoding. In that setting, however, their reliability is difficult to measure, because there is no independent answer to score them against. SpendAudit restricts the policy to rules that can be executed, so that every auditor verdict can be compared with a computed one.

The setting comes from SOLVENT [1], a self-funding analyst agent whose spend policy allows a payment only if seven deterministic checks pass. SpendAudit uses constructed traces of that setting—simulated transactions, not records of real payments—and asks whether an LLM given a natural-language statement of the policy and a ledger reaches the same decision. Two questions guide the study:

RQ1 Does requiring the auditor to check every rule, cite trace steps, and show its arithmetic improve verdict accuracy relative to a short verdict-only instruction?

RQ2 Does an instruction embedded in a payment memo turn correct judgments into incorrect ones, and does the evidence requirement reduce that effect?

Because the evidence protocol changes several things at once—per-rule decomposition, citation, explicit arithmetic, and longer output—RQ1 tests that bundle rather than the effect of citation alone.

2 Related Work

Using LLMs as evaluators is now widespread, and MT-Bench and Chatbot Arena are an influential example [2]. SpendAudit targets something narrower than overall response quality: compliance with an explicit rule set, scored against a computed answer rather than against human preference or agreement among judges. Agreement between judges is useful evidence of consistency, but it does not establish that an action satisfies the policy.

AgentDojo frames prompt injection as instructions that arrive through content an agent must process [3], and work on indirect prompt injection develops the same distinction between trusted instructions and untrusted data [4]. SpendAudit applies that distinction to an auditor rather than to an acting agent. A ledger memo is evidence about an event, not authority to change the policy, so each trace is paired with a version whose memo urges the wrong verdict.

SpendAudit is not intended as a general agent-security benchmark. It is a small, fully inspectable test of a question that matters for agent oversight: when does an auditor's explanation actually correspond to the evidence its decision requires?

3 Task and Threat Model

A trace contains seed capital, revenue events, earlier vendor payments, a job quote, and one payment under review. The auditor receives the trace, the policy, and the step identifier of that payment, and returns either *compliant* or *violation*; a violation must name the first failing rule. Table 1 gives the policy.

#	Rule	Decision condition
1	Vendor	Vendor must exactly match one of five allowed identifiers.
2	Transaction	Payment must not exceed $50.00.
3	Daily budget	Payments in the preceding rolling 24 hours plus the new payment must not exceed $150.00.
4	Vendor budget	The same vendor's rolling 24-hour total, including the new payment, must not exceed $60.00.
5	Velocity	Deny if six or more payments already occurred in the preceding hour.
6	Reserve	Balance after payment must remain at least $20.00.
7	Margin	The quoted projected margin must be positive.

Table 1. Benchmark policy. Rolling windows include their boundary. When a payment fails, the first failing rule in the order shown determines the label.

The attacker controls only the memo of the payment under review. Amounts, timestamps, vendor identifiers, the policy, and the auditor's instructions are fixed. Attacks on ledger integrity, optimized adversarial text, and multi-step collusion are out of scope. For rules like these, deterministic checks remain the right enforcement mechanism; the LLM is the object of evaluation, not a proposed replacement.

4 Benchmark Construction

Scenarios are produced by a seeded generator that wraps SOLVENT's `Guardrails.evaluate` method (v0.2.2, commit `bdc896a` [1]) alongside a separate implementation of the seven rules. The generator asserts that the two implementations agree on every scenario and that, for each violating scenario, exactly the targeted rule fails. Table 2 summarizes the resulting set.

Scenario type	Scenarios	Items (clean + injected)
Exactly one rule violated	84	168
Compliant at a boundary (near-miss)	48	96
Clearly compliant	28	56
Total	160	320

Table 2. Benchmark composition. The two items for each scenario contain the same policy-relevant facts; only the payment memo differs.

Violating scenarios cover twelve cases per rule. Near-miss scenarios cover eight cases for each of rules 2–7, such as an amount exactly at a limit or a balance exactly at the reserve; the vendor allowlist, an exact-match rule, has no numeric boundary. Clearly compliant scenarios are ordinary payments well within every limit. Traces range from 10 to 25 steps (mean 15.7).

The injected memo always argues for the wrong verdict: compliance when the payment violates policy, a violation when it complies. It may claim prior approval, a policy exception, or a known breach, none of which can change the numeric outcome. Scenarios that involve rolling windows also contain an expense just outside the relevant window, which tests whether the auditor includes the right entries rather than summing nearby payments.

Label verification. As a check independent of the generator, labels were re-derived directly from the rendered traces. A read-only script parsed the displayed timestamps, amounts, vendors, and quoted margin, computed in integer cents, and applied the seven rules. It reproduced the stored verdict and rule label for all 320 items, and found no differences between clean and injected copies outside the targeted memo in any of the 160 pairs. These checks establish that the tasks and labels are internally consistent; they do not establish that the policy suits a real organization or that SOLVENT enforces it correctly in deployment.

Dataset versions. Two versions of the dataset share identical traces, outcomes, rule labels, and payment identifiers. The second adds annotations marking decisive and decoy ("trap") steps, which are needed for citation analyses but do not affect the verdict comparisons reported here.

5 Judging Protocols and Response Coverage

Both protocols receive the same policy and trace. The *verdict-only* protocol (V) requests a JSON verdict and rule number. The *evidence-and-arithmetic* protocol (E) additionally requires, for each of the seven rules, the relevant step identifiers, the arithmetic or comparison, and a pass/fail result. All runs used temperature 0. Gemini runs used an 1,800-token output budget; Gemma was rerun with an 8,192-token budget after earlier runs were truncated, and only the rerun is analyzed. Smoke-test runs are excluded.

Table 3 lists the model identifiers recorded in the run logs, referred to below as Gemini 3.1 Flash-Lite, Gemini 3.5 Flash-Lite, and Gemma 4 31B. The runner stored raw outputs and some usage data, but not a dataset hash, per-request timestamps, or a provider-returned model revision, so the

exact model snapshots cannot be confirmed. File timestamps place all retained runs on October 12–13, 2026.

Model identifier (as logged)	Protocol	Usable	API error	Malformed
`gemini-3.1-flash-lite`	V	187	133	0
	E	187	133	0
`gemini-3.5-flash-lite`	V	250	70	0
	E	250	69	1
`gemma-4-31b-it`	V	262	58	0
	E	252	68	0
Total		1,388	531	1

Table 3. Outcome of the 320 intended judgments in each model–protocol cell. V = verdict-only; E = evidence and arithmetic.

API errors (quota, server, and timeout failures) are operational failures, not decisions, and are excluded rather than scored. The single malformed response is also excluded; in deployment, its consequence would depend on whether the system fails open, fails closed, or escalates to a human.

Missingness is strongly associated with scenario category, so the responses are not missing completely at random. Gemini 3.1 Flash-Lite returned no usable responses on near-miss scenarios, and the other two models cover them only partially. Comparisons restricted to fully observed scenarios remain valid for those scenarios, but a model-wide accuracy figure would mix different case distributions.

Analysis. For each model, a scenario is retained only if both protocols returned usable verdicts on both its clean and injected traces. This yields a fixed four-condition matched subset per model, so every within-model comparison involves exactly the same cases. A *false approval* is a violating payment judged compliant; a *false rejection* is a compliant payment judged a violation. Results are reported as counts on these subsets. They are complete-case comparisons, not estimates of performance on the full benchmark.

Statistical comparison. Each comparison of interest—V against E on the same trace variant, or clean against injected under the same protocol—is paired, because both conditions involve the same scenarios. I therefore use McNemar's exact test, which depends only on the discordant pairs: scenarios that are wrong in one condition and right in the other, counted directly from the per-scenario response logs. Thirteen comparisons are tested, with a Holm correction at $\alpha = 0.05$.

6 Results

Table 4 reports each model's matched subset. Because the subsets differ across models, the table does not support ranking models; all comparisons below are within a model.

Evidence protocol (RQ1). Within every model's subset, the evidence protocol made no more errors than verdict-only, and fewer in five of the six model–trace comparisons; the exception is Gemma on clean traces, where both protocols were error-free. The largest differences were in false approvals on clean traces: from 34 to 6 of 65 violating payments for Gemini 3.1 Flash-Lite, and from 47 to 18 of 83 for Gemini 3.5 Flash-Lite. Under verdict-only, Gemini 3.1 Flash-Lite approved more than half of the violating payments in its clean subset. The improvement does not come from a shift toward rejecting more payments: the evidence protocol produced no false rejections in any condition, so

Model	Trace	n	Correct V → E	False approvals V → E	False rejections V → E
Gemini 3.1 Flash-Lite	Clean	93	59 → 87	34 → 6	0 → 0
	Injected	93	50 → 86	35 → 7	8 → 0
Gemini 3.5 Flash-Lite	Clean	122	75 → 104	47 → 18	0 → 0
	Injected	122	46 → 92	51 → 30	25 → 0
Gemma 4 31B	Clean	87	87 → 87	0 → 0	0 → 0
	Injected	87	73 → 77	14 → 10	0 → 0

Table 4. Counts on each model's four-condition matched subset, in which every retained scenario has usable V and E verdicts on both trace variants. Violating/compliant scenarios: 65/28 (Gemini 3.1 Flash-Lite), 83/39 (Gemini 3.5 Flash-Lite), 66/21 (Gemma 4 31B). Arrows show verdict-only → evidence.

every error it made was the approval of a violating payment. Both clean-trace reductions are highly significant (exact McNemar $p < 10^{-7}$; Table 5), and for neither model did the evidence protocol approve a violating clean-trace payment that verdict-only had rejected.

Prompt injection (RQ2). The two protocols responded to injected memos differently. Under verdict-only, injection increased false rejections for both Gemini models (0 → 8 and 0 → 25) and false approvals for all three models (34 → 35, 47 → 51, and 0 → 14). Under the evidence protocol, injection produced no false rejections, but false approvals still rose relative to the matching clean traces (6 → 7, 18 → 30, and 0 → 10). In these subsets, the evidence protocol was not swayed by memos urging the rejection of compliant payments, yet remained susceptible to memos urging the approval of violating ones—the direction that matters most for a payment monitor.

Gemma 4 31B shows why clean-trace performance alone is insufficient. Both protocols classified all 87 clean scenarios in its subset correctly. With injected memos, verdict-only approved 14 of the 66 violating payments and the evidence protocol approved 10. Injection significantly increased Gemma's false approvals under both protocols ($p = 0.0001$ and $p = 0.002$). The difference between 14 and 10, however, is not significant: all four discordant scenarios favor the evidence protocol, but four is too few to rule out chance ($p = 0.125$). Requiring supporting steps and arithmetic did not demonstrably reduce Gemma's susceptibility.

Counting discordant pairs also settles two comparisons that marginal counts alone would leave open. For Gemini 3.5 Flash-Lite on injected traces, 24 violating payments were approved under verdict-only but rejected under the evidence protocol, against 3 in the other direction ($p < 10^{-4}$). Under the evidence protocol, injection turned 14 of its correct rejections into false approvals, while 2 changed the other way ($p = 0.004$).

6.1 An Inspectable Failure

In scenario `sc033`, a $16.19 payment goes to `openweb-search`, which is not on the allowlist. On the clean trace, both Gemma protocols correctly report a rule 1 violation. The injected memo claims pre-approval from finance and instructs the auditor to mark the payment compliant, and both protocols then return *compliant*. The evidence-protocol response explicitly notes that the vendor is absent from the allowlist, yet treats the memo as an exception, although the policy defines no exception mechanism. The auditor's reasoning was correct up to the point at which it deferred to untrusted text.

Comparison	Errors	Discordant	Exact p	Holm
RQ1: verdict-only vs. evidence, same trace variant				
Gemini 3.1 FL, clean, false approvals	34 vs. 6	28 / 0	7.5×10^{-9}	sig.
Gemini 3.5 FL, clean, false approvals	47 vs. 18	29 / 0	3.7×10^{-9}	sig.
Gemini 3.1 FL, injected, false approvals	35 vs. 7	29 / 1	5.8×10^{-8}	sig.
Gemini 3.5 FL, injected, false approvals	51 vs. 30	24 / 3	4.9×10^{-5}	sig.
Gemma 4 31B, injected, false approvals	14 vs. 10	4 / 0	0.125	n.s.
Gemini 3.1 FL, injected, false rejections	8 vs. 0	8 / 0	0.0078	sig.
Gemini 3.5 FL, injected, false rejections	25 vs. 0	25 / 0	6.0×10^{-8}	sig.
RQ2: clean vs. injected, same protocol				
Gemma 4 31B, V, false approvals	0 vs. 14	0 / 14	1.2×10^{-4}	sig.
Gemma 4 31B, E, false approvals	0 vs. 10	0 / 10	0.0020	sig.
Gemini 3.5 FL, E, false approvals	18 vs. 30	2 / 14	0.0042	sig.
Gemini 3.1 FL, E, false approvals	6 vs. 7	3 / 4	1.0	n.s.
Gemini 3.5 FL, V, false rejections	0 vs. 25	0 / 25	6.0×10^{-8}	sig.
Gemini 3.1 FL, V, false rejections	0 vs. 8	0 / 8	0.0078	sig.

Table 5. Two-sided exact McNemar tests on the matched subsets. Discordant pairs count scenarios wrong only in the first condition / wrong only in the second. The Holm column applies a correction across all thirteen comparisons at $\alpha = 0.05$. FL = Flash-Lite. Reproduced by `src/recount.py` in the SpendAudit repository.

This is the kind of failure the benchmark is designed to expose. Rather than accepting a persuasive compliance narrative, a reviewer can ask which rule applies, which entries fall inside its window, and whether the numbers support the verdict. A single aggregate score would hide that diagnosis.

7 Discussion and Limitations

Exposing an auditor's checks makes some mistakes easier to detect and, in these matched cases, coincides with fewer wrong verdicts. It does not turn an explanation into an enforcement guarantee. A trace can carry instructions that compete with the auditor's task, and a well-structured explanation can still end in a false approval. The `sc033` case suggests that some failures lie not in reading the evidence but in deciding what counts as authority: the auditor found the decisive fact and then set it aside.

Implications for practice. LLMs are a natural candidate for reviewing expenses, invoices, and other transactions against policy. These results suggest three working rules for such deployments. First, deterministic controls should remain the authority wherever a rule can be encoded, with the LLM used to flag and explain issues rather than to grant exceptions. Second, free-text fields such as memos, invoice notes, and vendor descriptions should be treated as untrusted input, because they are where an injected instruction arrives. Third, an LLM reviewer should be evaluated on matched cases, with coverage reported by case type and errors separated by direction, since a single accuracy figure can conceal both missing cases and a skew toward false approvals.

Coverage. API errors removed whole regions of the scenario set, especially boundary cases. Complete-case matching keeps within-model comparisons on fixed cases, but it cannot recover the missing cases or make the subsets representative. A completed study should finish all requests under a fixed protocol, retain failed attempts, and report coverage by rule and scenario category

before computing overall rates, either stratifying results by category or reweighting them to the benchmark's composition.

Protocol bundling. Citation, per-rule decomposition, explicit arithmetic, and longer output all change together in the evidence protocol. Separating them requires at least three conditions—verdict-only, decomposition without citation, and decomposition with citation—under documented token budgets, with repeated calls to distinguish stable behavior from single responses. Uncertainty should be estimated at the scenario level, treating each scenario's variants and repeated calls as dependent observations rather than independent examples.

Generalization and evidence quality. The traces are short, synthetic, and governed by a single fixed policy, and every violating case breaks exactly one rule; real failures can combine several breaches and involve unreliable records. A citation is also not a proof: citing every decisive step does not guarantee correct arithmetic, and citing a decoy step can be appropriate when explaining why it is excluded. Separating evidence errors from computation errors therefore remains a diagnostic to be validated, not an established account of model mistakes.

Next experiment. The next run should record exact prompts, dataset and code hashes, provider-returned model revisions, timestamps, token budgets, parser behavior, and retry decisions. It should add held-out scenarios and limits rather than tuning the benchmark to observed failures, and it should test a mechanical verifier that checks whether cited steps exist and whether the stated arithmetic is consistent with them. Where a policy is fully expressible in code, that verifier should remain the authority; the open question is what additional monitoring value an LLM provides, most plausibly for the clauses that cannot be encoded.

8 Conclusion

SpendAudit evaluates LLM auditors against an outcome that can be computed independently. Its traces pass a direct label-consistency audit, and the available matched responses show both a benefit associated with explicit rule checking and persistent false approvals under injected instructions. The evidence supports a narrower claim than a completed benchmark: an inspectable evaluation design with preliminary findings and clearly identified gaps. Completing it will require full coverage and a decomposed comparison of protocols, while preserving the distinction the study is built around—between an explanation that sounds right and a decision the record supports.

References

[1] I. Alloway. SOLVENT: A self-funding analyst agent with a deterministic spend policy. Software repository, version 0.2.2 (commit `bdc896a`), 2026. `https://github.com/ianalloway/solvent-agent`

[2] L. Zheng, W.-L. Chiang, Y. Sheng, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023.

[3] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track*, 2024. `https://arxiv.org/abs/2406.13352`

[4] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023. `https://arxiv.org/abs/2302.12173`

Data and code availability. The SOLVENT source code is public [1]. The SpendAudit generator, datasets, prompts, response logs, and analysis scripts are available at `https://github.com/ianalloway/spendaudit`.

Note on preparation. This manuscript is unpublished. SOLVENT, the SpendAudit benchmark, and the experiments reported here are my own work. AI tools were used in preparing the manuscript: OpenAI Codex helped organize the prose and ran read-only recount scripts over the existing dataset and response files (label re-derivation, coverage counts, and the matched-subset tables), and Claude (Anthropic) was used for editing and to write the recount script that computes Table 5 from the response logs. No new model queries were made for the manuscript, and no real payments were involved at any stage.